

Assignment 1: Builder Pattern

Study report with one Builder

Astana IT University | ShP-2216 - Software Design Patterns | Java 17

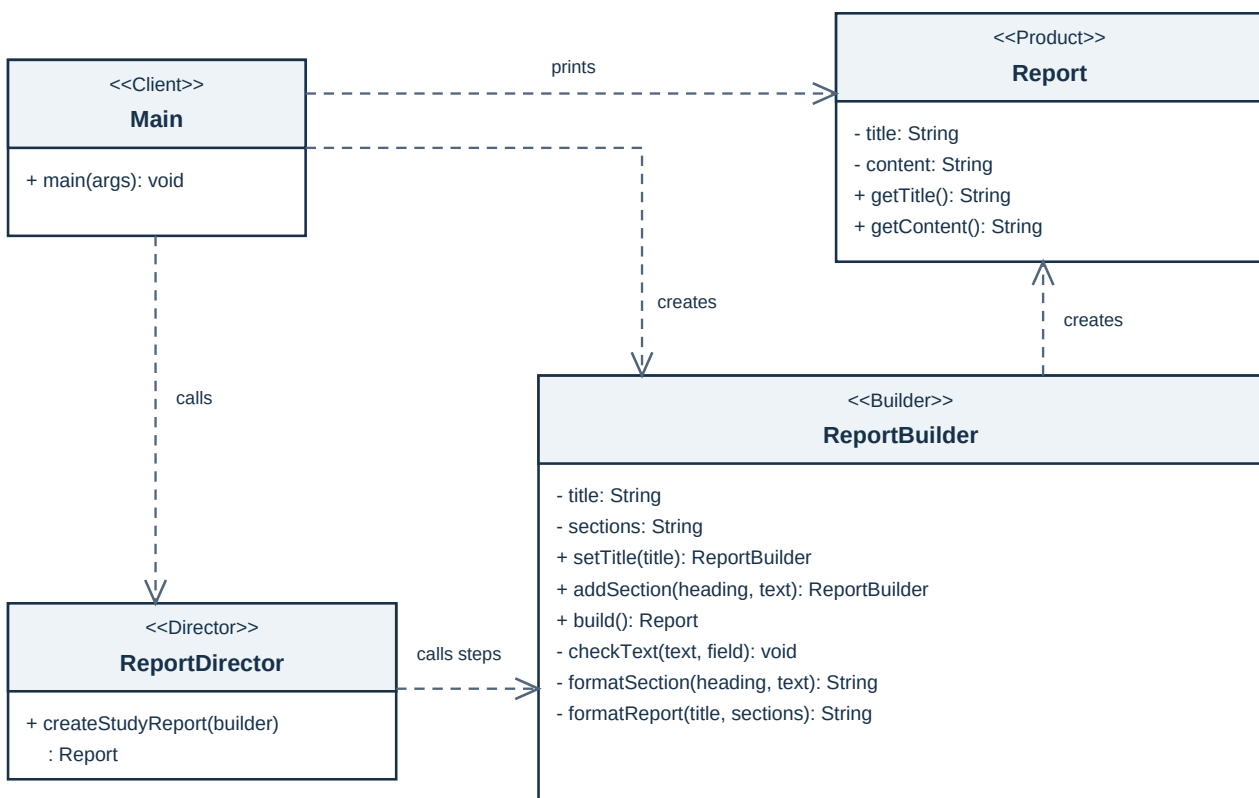
1. Introduction

The product is a study report with a title and several sections. Each section has a heading and some text. Builder lets the program add these parts step by step and create the finished Report by calling build().

There is one ReportBuilder class. It stores the title and sections and formats them as plain text. ReportDirector contains a ready-made sequence of steps. Main shows both the Director recipe and a custom report built with a chain of calls.

Report is a normal class with a constructor, two private fields and getters. ReportBuilder is also a normal class; it can be created directly with new ReportBuilder(). The example uses basic classes, methods, strings and conditions. It does not use inheritance.

2. UML class diagram



Dashed arrows show use or creation. Main creates a ReportBuilder, calls the Director and prints the finished Report. ReportDirector calls the construction steps. ReportBuilder stores the parts and creates the product. The editable diagram source is docs/uml.puml.

3. Clean Code examples

1. Meaningful names

Names describe report-writing actions. Reading this chain shows what will be built without needing extra comments. Source: Main.java.

```
.setTitle("My homework")
.addSection("Task", "Practice method chaining")
.build();
```

2. Small methods

Each formatting method has one task. This method formats only one plain text section; formatReport() adds the report title. Source: ReportBuilder.java.

```
private String formatSection(String heading, String text) {
    return heading + "\n" + text + "\n\n";
}
```

3. Reuse validation logic

The same checkText() helper checks the title, heading and section text. The null and blank checks are written once, instead of repeating them for every field. Source: ReportBuilder.java, excerpt from addSection().

```
checkText(heading, "Heading");
checkText(text, "Section text");
sections += formatSection(heading, text);
return this;
```

4. Validate the result

build() gives a clear error when required parts are missing. There is a similar check for missing sections. Source: ReportBuilder.java.

```
if (title.isBlank()) {
    throw new IllegalStateException(
        "Add a title before building the report");
}
```

5. Keep fields private

Other classes read a finished report through getters. They cannot directly replace its fields. final means these fields are assigned in the constructor and cannot be reassigned. Source: Report.java.

```
private final String title;
private final String content;

public String getContent() {
    return content;
}
```

4. Running the program

Open the project in IntelliJ IDEA, select JDK 17, mark src/main/java as Sources Root and run Main.main(). Alternatively, compile and run with the commands below. The program prints a study report and a custom report built without a Director.

```
javac --release 17 -d build src/main/java/kz/aitu/builder/*.java
java -cp build kz.aitu.builder.Main
```

The program was compiled and run with Amazon Corretto 17.0.13. The console displayed the title and sections for both reports. The project uses no external Java libraries.

A typical call with a Director is shown below. The Director sets the title and adds three sections before calling build().

```
ReportDirector director = new ReportDirector();
Report report = director.createStudyReport(
    new ReportBuilder());
System.out.println(report.getContent());
```

5. Conclusion

Builder makes construction steps easy to read. A client adds sections one by one and then calls build() to receive the completed Report. The Director can repeat a known sequence of steps. A custom report can be assembled without the Director.

Keeping just one builder makes the code shorter and easier to follow. The tradeoff is that it produces only one representation and does not demonstrate interchangeable concrete builders. String concatenation is sufficient for this small example, although large documents would need a more efficient approach.

For a report whose content is already prepared, an ordinary constructor would be simpler. Builder is useful here because the client adds sections gradually. Each new report uses a fresh builder because build() does not reset stored data.

Scope difference: the original assignment requires an interface or abstract Builder and at least two ConcreteBuilders producing different representations. This version intentionally contains one ordinary ReportBuilder and does not meet those requirements.

6. Repository

GitHub: [unknown9error/aitu-software-design-pattern-assignment1](https://github.com/unknown9error/aitu-software-design-pattern-assignment1)

The repository contains source code, a README, incremental commits, UML and this report. This PDF is a separate written report, not a feature of the Java program.